VedaAI: Master Implementation Blueprint v2.0

Complete Phase-by-Phase Engineering Execution Plan with Full Reasoning, Alternatives, Consequences & Go/No-Go Criteria

> **IMPORTANT**
>
> This is **Version 2.0** of the blueprint. All 7 identified gaps from the v1.0 self-review have been applied. Changes are applied in cascade-safe order. Every cross-reference, phase number, test case, and architecture diagram has been reverified for internal consistency.

What Changed From v1.0 and Why

Change #	What Changed	Why	Cascade Risk Mitigated
C1	User Profile Schema added to Phase 3; Entity Extractor merges profile+query	Ayurveda is personalized medicine — stateless queries miss chronic conditions	Tests updated with profile-aware variant
C2	Fallback Hierarchy added when Neo4j returns 0 results	System silently hallucinated when herbs were outside seed dataset	Proponent/Opponent prompts updated; Test 2.D added
C3	New Step 2.3.5: Cypher Query Builder with fuzzy matching + synonym resolution	Raw extracted names broke exact Cypher string matches	Step 2.3 input spec updated; Test 2.A now verifies synonym resolution
C4	Safety Violation Rate + Clinical Relevance Score added to Phase 5	BLEU/ROUGE don't measure health safety; reviewer would reject	Phase 5 dataset design updated to include planted contraindications
C5	LangGraph StateGraph replaces RunnableSequence from the start	Conditional routing (early-exit) is unreadable in RunnableSequence	Judge early-exit is now a conditional edge, not code logic
C6	Prompt Versioning directory + LangSmith setup before first LLM call	20-30 prompt iterations are inevitable; without versioning you lose working prompts	Positioned as Step 2.0 before any agent step
C7	Phase reordered: Full-Stack → Phase 3, Vision → Phase 4	Vision has hard dependency on Express file-upload endpoint (Phase 3)	All cross-references renumbered; Go/No-Go tree updated

Preface: Why Phasing Matters More Than Speed

You cannot recover from a broken foundation at the integration stage.

If you skip Phase 1 (the data layer) and jump straight to the Next.js chat UI, you will eventually hit the point where the AI microservice returns bad data, and you will not know whether the bug is in your graph schema, your vector embedding strategy, your prompt engineering, or your frontend API call. You will spend days debugging an integration of systems that were never individually verified.

The phases are ordered to eliminate that risk. Each phase produces a **self-contained, independently testable artifact**. You do not proceed to the next phase until the current phase passes its Go/No-Go criteria.

New in v2.0: The phase order has been corrected based on dependency analysis:

- Phase 3 is now Full-Stack (Express + Next.js) — because the Vision Agent needs the file upload endpoint.
- Phase 4 is now the Vision Agent — it now has everything it depends on already built.

PHASE 1: The Dual-Knowledge Store (The Data Layer)

What this phase IS and IS NOT

IS: Building and verifying the two data stores — Neo4j (structured relational rules) and ChromaDB (unstructured textual context).

IS NOT: Any agent, any API endpoint, any UI. Not a single LangChain line. Not a single LLM call. This phase is pure data engineering.

Step 1.1 — Ontology Design for the Knowledge Graph

What this step does

Decides what entities exist in the domain, what relationships connect them, and what properties each node and relationship carries. This is the schema design of the graph database. Everything downstream depends on getting this right.

The Full Ontology

Node Types and their properties:

```
(:Dosha)          → name (Vata / Pitta / Kapha), season_dominance, body_system
(:Herb)           → name, sanskrit_name, botanical_name, rasa (taste), guna (quality),
                    virya (potency: Ushna=hot / Sheeta=cold), vipaka (post-digestive
effect),
                    prabhava (special effect), contraindications_text,
                    synonyms[] ← NEW: ["Adrak","Shunti","Zingiber officinale"] for Ginger
(:Symptom)        → name, dosha_association, body_system, synonyms[]
(:Disease)        → name, ayurvedic_name, dosha_imbalance, severity
(:Season)         → name (Vasanta/Grishma/Varsha/Sharad/Hemanta/Shishira),
                    dominant_dosha, diet_recommendations
(:Remedy)         → name, type (food/herbal/lifestyle), preparation_method
(:Formulation)    → name, brand_name, constituent_herbs[], form (churna/kwath/capsule)
(:Condition)      → name (Pregnancy/Hypertension/Liver Disease/Pediatric),
severity_modifier

Relationship Types:
[:PACIFIES]            → strength (1-5), condition_required
[:AGGRAVATES]          → strength (1-5), condition_required
[:TREATS]              → efficacy_level, source_text
[:CONTRAINDICATED_FOR] → severity (mild/moderate/severe), reason
[:COMPOSED_OF]         → quantity, role
[:DOMINANT_IN]         → degree (mildly/moderately/severely)
[:WORSENS_IN]          → season_name
[:ASSOCIATED_WITH]     → clinical_basis
```

> NOTE
>
> The `synonyms[]` property on `:Herb` and `:Symptom` nodes is **new in v2.0**. It is required by the Cypher Query Builder (Step 2.3.5) for fuzzy matching. Populate synonym arrays when loading the seed dataset. The `:Condition` node type supports the User Profile integration (Phase 3).

Why THIS ontology and not a simpler one?

Alternative A (Just Herb + Dosha nodes): You can answer "*Does Ginger aggravate Pitta?*" but you **cannot** answer "*Is Ginger contraindicated for someone with acid reflux in summer?*" — because that 3-hop traversal requires Season nodes, Symptom nodes, and the `[:ASSOCIATED_WITH]` relationship. That 3-hop traversal **is literally the entire research contribution of your paper.** If the ontology cannot support it, you have no paper.

Alternative B (Full Sanskrit ontology — all 20 Gunas, 8 Rasas, Samprapti chains): 2 months of ontology design alone. You will never finish the system.

Why the proposed 8-node, 8-relationship ontology is the correct middle path:

- Complex enough for multi-hop traversal (the research contribution).
- Simple enough to populate with 2 people in 3–4 days.
- Every node type maps directly to Chapter 5.2.2 of your synopsis.

Elicitation: Source of Truth Priority Order

1. **Charaka Samhita** — highest authority for herb-dosha relationships.
2. **AYUSH Ministry Published Guidelines** — government-verified, safest to cite.
3. **AYU Journal / Journal of Ethnopharmacology** — peer-reviewed.
4. **GRAYU/DBpedia Ayurvedic datasets** — semi-structured, verify before use.

Data Quality Gate: A relationship is only added to Neo4j if it appears in at least 2 of the above sources. This gate is what separates your system from a hallucinating LLM.

Step 1.2 — Deciding the Seed Dataset Size

Option A — 10 herbs (Minimal): Cannot run 150 unbiased benchmark queries. Evaluation dataset looks artificially curated. Reviewers will reject the paper.

Option B — 500+ herbs (Maximal): 3–4 months of data entry. Project collapses.

Option C — 50–75 herbs, 3 doshas, 6 seasons, ~40 symptoms, ~30 diseases, ~15 conditions (Chosen):

- 3–4 days to populate.
- 50 herbs × 4 relationship types = 200+ edges minimum. Multi-hop traversal supports thousands of distinct reasoning paths.
- Covers all 150 benchmark queries (designed in Phase 5 to use only this seed set).

> IMPORTANT
>
> When populating the seed dataset, also populate the `synonyms[]` array on every Herb node. Failure to do this means the Cypher Query Builder (Step 2.3.5) cannot perform synonym resolution, causing zero-result failures for queries that use regional herb names.

Step 1.3 — Neo4j Deployment Strategy

Neo4j Desktop (Local): Full control, visual graph browser. Risk: database lost if laptop fails before viva.

Neo4j Aura Free Tier (Cloud) + Local `.cypher` backup (Chosen):

- Aura is always accessible regardless of laptop state.
- `.cypher` backup file: if internet fails at the viva venue, you can spin up a local instance in 5 minutes and import the script.
- Redundancy is not paranoia — it is professional engineering practice.

Docker Container: Most reproducible, but most complex setup. Port mapping errors waste time. Not justified for a 4-person student project.

Step 1.4 — ChromaDB Vector Store Setup

Text Sources

- AYUSH Ministry clinical guideline PDFs (public domain, legally safe).
- Open-domain translations of Charaka Samhita and Ashtanga Hridayam.
- AYU Journal open-access research abstracts.

Do NOT ingest copyrighted books. Using unlicensed copyrighted material in a published paper creates legal and ethical problems.

Embedding Model

OpenAI `text-embedding-3-small`: Excellent general English quality, but not optimized for Sanskrit transliteration terms. Cost: ~$0.40 to embed 10,000 chunks. Not chosen.

Google `text-embedding-004` (Chosen): Strong multilingual support, handles transliterated Sanskrit better. Free tier: 1,500 requests/minute. Consistent with the Gemini API keys used throughout the system.

`sentence-transformers/all-MiniLM-L6-v2` (Local open-source): Decent English quality but not trained on Ayurvedic domain text. Semantic search quality drops noticeably on domain-specific queries, confounding the benchmark results.

Why embedding model matters for benchmarking: If you use a weak embedding model, your Baseline RAG already performs poorly, making the graph's contribution look artificially large. A reviewer will flag this as confounding variable bias.

Chunk Size

128-token chunks: Too small. Ayurvedic text describes herb properties across 3–4 sentences. Mid-sentence cuts produce incoherent embeddings.

1024-token chunks: Too large. Excessive irrelevant context dilutes signal quality in the Proponent Agent's window.

512-token chunks with 50-token overlap (Chosen): The 50-token overlap ensures sentences split at chunk boundaries are fully captured in at least one chunk. Critical for Ayurvedic property descriptions that span sentences.

Step 1.5 — Phase 1 Verification: Go/No-Go Tests

ALL tests must pass before Phase 2 begins.

Test 1.A — The Multi-Hop Cypher Test

```
cypher
MATCH (s:Season {name: 'Grishma'})-[:AGGRAVATES]->(d:Dosha {name: 'Pitta'})
MATCH (h:Herb)-[:AGGRAVATES]->(d)
MATCH (sym:Symptom)-[:ASSOCIATED_WITH]->(d)
WHERE sym.name = 'Acid Reflux'
RETURN h.name AS Contraindicated_Herb, d.name AS Aggravated_Dosha,
       s.name AS Current_Season
```

Pass condition: Returns `Ginger` and `Trikatu` in under 200ms. If this passes, the graph is correctly populated AND multi-hop traversal works.

Test 1.B — The Synonym Resolution Test *(New in v2.0)*

```
cypher
MATCH (h:Herb) WHERE 'Adrak' IN h.synonyms OR h.name = 'Adrak'
RETURN h.name AS canonical_name, h.synonyms AS known_by
```

Pass condition: Returns `{canonical_name: 'Ginger', known_by: ['Adrak','Shunti','Zingiber officinale']}`. This verifies synonym arrays are populated correctly for the Cypher Query Builder in Step 2.3.5.

Test 1.C — The Semantic Retrieval Test

Query ChromaDB: "*herbal remedies for inflammation and joint pain with Vata constitution*"

Pass condition: Top-3 chunks contain at least 2 anti-inflammatory herbs, at least 1 Vata Dosha mention, cosine similarity scores above 0.72.

Test 1.D — Schema Integrity Test

```
cypher
MATCH (n) RETURN labels(n) AS type, count(n) AS count
MATCH ()-[r]-() RETURN type(r) AS rel, count(r) AS count
```

Pass condition: Min 50 `:Herb` nodes (each with non-empty `synonyms[]`), 3 `:Dosha`, 6 `:Season`, 25+ `:Symptom`, 10+ `:Condition` nodes. Min 150 total relationships.

Phase 1 Go/No-Go Decision Rule

All four tests pass → proceed to Phase 2.

- Test 1.A fails → fix Cypher population scripts.
- Test 1.B fails → add synonym arrays to Herb nodes before anything else.
- Test 1.C fails → reconsider embedding model or chunk size.
- Test 1.D fails → more data needed.

If you skip these tests and enter Phase 2: The Hybrid Retriever returns empty graph results for common queries. You spend days debugging the agent pipeline when the real problem is missing data. You cannot distinguish agent failures from data failures.

PHASE 2: The Multi-Agent Verification Pipeline (The AI Engine)

What this phase IS and IS NOT

IS: Building the complete 6-component LangGraph pipeline: Entity Extractor → Canonical Name Lookup → Hybrid Retriever → Proponent Node → Opponent Node → Judge Node. The core research contribution.

IS NOT: Any UI, any Express API, any image processing. This phase runs entirely from terminal or a Jest/pytest test harness. Input: text string + optional user profile JSON. Output: structured JSON object.

Step 2.0 — Infrastructure Before Any LLM Call: Prompt Versioning + LangSmith *(New in v2.0)*

Why This Step Exists (And Why It Comes Before Everything Else)

Prompt engineering for Proponent/Opponent/Judge is **iterative**. You will change your prompts 20–30 times before they reliably output valid structured JSON every single time, correctly weight graph facts over text chunks, and handle all edge cases gracefully.

Without versioning, you will accidentally overwrite a working prompt during an experiment, with no way to recover it. This has killed more student AI projects than any technical bug.

Repository Structure to Create NOW

```
/prompts
  /entity_extractor
    v1_basic.txt
  /proponent
    v1_basic.txt
  /opponent
    v1_basic.txt
  /judge
    v1_basic.txt
```

Increment the version number every time you make a meaningful change to any prompt. Never delete old versions.

LangSmith Setup

LangSmith (free tier at smith.langchain.com) logs every LLM call automatically when you set:

```
bash
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_key
LANGCHAIN_PROJECT=vedaai-dev
```

Every call is logged with: prompt version, input, output, latency, token count, and model used. This data goes directly into your paper's methodology section as evidence of systematic evaluation.

> IMPORTANT
>
> Set up LangSmith BEFORE writing a single agent. If you add it after, you lose all the debugging data from your early iterations — the most valuable data for understanding why your prompts behaved the way they did.

Step 2.1 — Choosing the Orchestration Framework

Decision: LangGraph StateGraph (Chosen from the Start)

Why NOT LangChain's `RunnableSequence` (v1.0 recommendation — now corrected): The debate pipeline has a critical conditional branch: "*If Opponent verdict is VERIFIED, skip the Judge synthesis and go directly to output formatter.*" Implementing this conditional branch in `RunnableSequence` requires `RunnableBranch`, which becomes unreadable at 5+ agents. You will spend 2 days fighting the framework and then migrate to LangGraph anyway.

Why NOT Raw Async/Await (No framework): You reinvent retry logic, streaming, rate limiting, and structured output parsing. For a 6-node pipeline with branching logic, raw async code becomes 800+ lines of untestable spaghetti.

Why NOT Python LangChain: Your paper specifies LangChain JS. Your frontend is Next.js. Introducing a Python microservice adds a cross-language HTTP boundary and a second deployment to manage.

LangGraph StateGraph (Chosen):

- Explicit state machine: nodes, edges, and conditional edges.
- The early-exit path (Opponent → output formatter, bypassing Judge) is expressed as a single conditional edge. Clean, readable, testable.
- LangGraph integrates natively with LangSmith for full trace visibility.
- The LangChain documentation explicitly recommends LangGraph for multi-agent pipelines.

LangGraph State Schema

The state object that flows between all nodes:

```
typescript
interface VedaAIState {
  // Input
  raw_query: string;
  user_profile: UserProfile | null;       // From Phase 3 MongoDB

  // Step 2.3.5 output
  canonical_entities: CanonicalEntities;

  // Step 2.3 output
  vector_chunks: VectorChunk[];
  graph_facts: GraphFact[];
  graph_available: boolean;               // NEW: false if Neo4j returned 0 results

  // Step 2.4 output (Proponent)
  proponent_draft: ProponentDraft;

  // Step 2.5 output (Opponent)
  opponent_verdict: "FLAGGED" | "VERIFIED";
  opponent_flags: OpponentFlag[];

  // Step 2.6 output (Judge)
  final_response: FinalResponse;
  confidence_score: number;
}
```

Every node reads from and writes to this state object. No node calls another node directly — the graph edges define the execution order.

Step 2.2 — The Entity Extractor Node

What it does

Takes the raw user query AND the user's stored health profile and produces a merged, structured entity set for the Cypher Query Builder.

> IMPORTANT
>
> **New in v2.0:** The Entity Extractor now merges `raw_query + user_profile` before entity extraction. In v1.0 it only processed the raw query. The change means that even if a user forgets to mention "I have hypertension," the extractor pulls it from the stored profile and includes it in the entity set. This is critical for safety.

Input:

```
json
{
  "raw_query": "Can I take Ashwagandha for stress?",
  "user_profile": {
    "prakriti": "Pitta-Vata",
    "chronic_conditions": ["Hypertension"],
    "age": 42,
    "pregnancy": false,
    "current_season": "Grishma"
  }
}
```

Output (structured JSON via Gemini Flash + Zod schema):

```
json
{
  "mentioned_herbs": ["Ashwagandha"],
  "symptoms": ["Stress", "Anxiety"],
  "dosha_state": "Pitta-Vata",
  "current_season": "Grishma",
  "patient_conditions": ["Hypertension"],
  "age": 42,
  "pregnancy": false,
  "query_intent": "herb_safety_check",
  "profile_merged": true
}
```

Implementation Decision: LLM + Zod Schema (Not Classical NLP)

Classical NLP (spaCy NER) cannot infer "*In July*" → *Grishma season* or merge implicit profile context. This requires world knowledge that only an LLM has.

Critical implementation detail: Use Gemini's structured output mode (enforced JSON schema), NOT free-text prompting. Free-text prompting will occasionally add explanatory text before/after the JSON, breaking your parser. Enforced schema mode returns parseable JSON 100% of the time.

Why Entity Extraction Is Not Optional

Entity extraction is the **sensory organ** of the pipeline. Wrong herb name → zero Cypher results → Proponent hallucinate without grounding → safety guarantee broken. Missing chronic condition → Opponent cannot check the contraindication → dangerous recommendation passes through.

Step 2.3.5 — The Cypher Query Builder *(New in v2.0)*

Why This Step Exists

In v1.0, the blueprint jumped directly from Entity Extractor to Hybrid Retriever, treating the Cypher query construction as a trivial one-liner. **It is not.**

The specific failure mode: User says "I have Vata problems." Entity Extractor outputs `dosha_state: "Vata"`. Your Cypher query does `MATCH (d:Dosha {name: "Vata"})`. This works. But if the Entity Extractor outputs `dosha_state: "predominantly Vata"` (a perfectly valid LLM paraphrase), the exact string match `{name: "predominantly Vata"}` returns **zero results**. The entire pipeline then runs on zero graph context and the Proponent hallucinates.

This class of failure accounts for approximately 30–40% of silent pipeline failures in production RAG systems.

What the Cypher Query Builder Does

1. **Normalize Dosha mentions:** Map any variant ("predominantly Vata," "Vata type," "Vata dominant") to canonical `"Vata"` via a simple lookup dictionary.
2. **Resolve herb synonyms:** For each herb name extracted, run:

   ```
   cypher
   MATCH (h:Herb)
   WHERE toLower(h.name) = toLower($name) OR toLower($name) IN [s IN h.synonyms |
   toLower(s)]
   RETURN h.name AS canonical_name LIMIT 1
   ```

   Replace the raw extracted name with the canonical name before any further queries.
3. **Normalize symptom names:** Map common English symptom descriptions to their graph equivalents ("heartburn" → "Acid Reflux," "tummy upset" → "Indigestion").
4. **Flag unresolvable entities:** If an herb name cannot be matched to any node (not in `name` and not in any `synonyms[]` array), mark it as `unresolved: true`. These go into the fallback path (Step 2.3).

Output of Cypher Query Builder:

```
json
{
  "resolved_herbs": [
    {"raw": "Adrak", "canonical": "Ginger", "resolved": true},
    {"raw": "XYZHerb", "canonical": null, "resolved": false}
  ],
  "resolved_symptoms": ["Acid Reflux", "Flatulence"],
  "resolved_dosha": "Pitta",
  "resolved_season": "Grishma",
  "resolved_conditions": ["Hypertension"],
  "has_unresolved": true
}
```

Consequence of skipping this step: ~30–40% of queries silently fail to find graph data. The system degrades to pure LLM hallucination for those queries. Your benchmark hallucination rate is artificially high, making your results look worse than they actually are.

Step 2.3 — The Hybrid Retriever Node

What it does

Takes the **canonical entities** (NOT raw extracted names) from the Cypher Query Builder and queries BOTH ChromaDB and Neo4j simultaneously.

Two Retrieval Streams

Stream A (ChromaDB — Probabilistic):

- Embeds the original user query.
- Retrieves top-K=5 semantically similar chunks.
- Returns: `[{chunk_text, source_document, similarity_score}]`

Stream B (Neo4j — Deterministic):

- Uses canonical resolved herb names, dosha, season, and condition names.
- Traverses the graph for all relevant relationships.
- Returns: `[{herb, relationship_type, target_node, relationship_properties}]`

CRITICAL NEW LOGIC: The Fallback Hierarchy *(New in v2.0)*

In v1.0, the blueprint was completely silent on what happens when Neo4j returns 0 results. This is a dangerous omission because the system would silently degrade to pure LLM hallucination for any query involving herbs outside the 50-herb seed set.

v2.0 Fallback Hierarchy (executed before calling the Proponent):

```
Neo4j Cypher query runs
    │
    ├── Results ≥ 1 node?
    │       → YES: graph_available = true. Full pipeline runs normally.
    │       → NO:  ↓
    │
    ├── Any unresolved_herbs? (from Cypher Query Builder)
    │       → YES: This is a scoping issue.
    │              Set graph_available = false.
    │              Set warning: "[herb] is outside our verified knowledge base."
    │              Continue with ChromaDB-only context.
    │       → NO (all herbs resolved but graph still empty):
    │              This is a graph population gap.
    │              Set graph_available = false.
    │              Log this query for future graph expansion.
    │              Continue with ChromaDB-only context.
    │
ChromaDB similarity scores checked:
    ├── Top-1 similarity > 0.65?
    │       → YES: ChromaDB-only context is usable. Proponent runs with text context only.
    │              Opponent is notified: graph_available = false.
    │       → NO (below 0.65):
    │              COMPLETE FALLBACK: Both knowledge sources insufficient.
    │              Return: "This query is outside our verified knowledge base.
    │                       Please consult a certified Ayurvedic practitioner (BAMS)."
    │              Pipeline terminates. No LLM generation.
```

Why the 0.65 ChromaDB threshold for complete fallback: Below 0.65, the retrieved text chunks are semantically too distant to be relevant context. Generating from them produces confidently-worded hallucinations. It is safer and more honest to refuse to answer than to hallucinate.

Merging the Two Retrieval Streams

Chosen approach — Two labeled sections in the prompt context:

```
=== VERIFIED KNOWLEDGE GRAPH FACTS ===
(Serialized graph edges — e.g., "Ginger AGGRAVATES Pitta with severity 4/5")
[If graph_available = false, this section reads: "No graph facts available for this
query."]

=== TEXTUAL KNOWLEDGE BASE CONTEXT ===
(ChromaDB chunk 1)
(ChromaDB chunk 2)
...
```

The Proponent's system prompt explicitly states: "*When the Verified Knowledge Graph Facts contradict the Textual Context, always defer to the Graph Facts. If no Graph Facts are available, flag all claims as [REQUIRES GRAPH VERIFICATION].*"

Step 2.4 — The Proponent Node

What it does

Generates an initial Ayurvedic recommendation from the merged context.

System Prompt Design

The Proponent's system prompt specifies exactly (prompt file: `prompts/proponent/v1_basic.txt`):

1. **Role:** "You are a knowledgeable Ayurvedic practitioner assistant."
2. **Priority rule:** Graph facts > textual context > general knowledge.
3. **Graph unavailability rule:** "If the Graph Facts section says 'No graph facts available,' mark ALL property claims about herbs with [REQUIRES GRAPH VERIFICATION]."
4. **Safety constraint:** "If you are uncertain about any claim, mark it with [UNVERIFIED]."
5. **Output structure:** Enforced JSON schema via Gemini structured output:

```
json
{
  "recommendation": "...",
  "reasoning": "...",
  "mentioned_herbs": ["..."],
  "claimed_properties": [
    {"claim": "Ginger aids digestion", "source": "graph" | "text" | "unverified"}
  ],
  "confidence_before_review": 0.XX,
  "disclaimer_needed": true | false,
  "graph_available": true | false
}
```

Why Structured Output from the Proponent Is Not Optional

If the Proponent outputs a paragraph, the Opponent must parse natural language to identify claims. This introduces imprecision. If the Proponent outputs a structured JSON with `"claimed_properties"` listing each claim with its source, the Opponent directly audits each claim against the graph. More accurate, more reliable, better debate outcomes.

Step 2.5 — The Opponent Node

What it does

Reviews the Proponent's structured draft looking specifically for:

1. Claims contradicted by Neo4j graph facts.
2. Missing contraindications (pregnancy, hypertension, liver disease, pediatric age).
3. Clinically unverified claims not backed by text context.
4. Dosha aggravation risks not mentioned in the draft.

The Opponent Must Be Given Re-Querying Capability *(Critical — unchanged from v1.0)*

Why: The Proponent may generate claims from its pre-training knowledge that were NOT in the initial retrieval context (because no text chunk matched them semantically). The initial context passed to the Opponent will not contain these claims. The Opponent must run its own targeted Cypher queries based on the Proponent's specific `claimed_properties`.

For each claim, the Opponent runs:

```
cypher
MATCH (h:Herb {name: $canonical_herb})-[r]->(n)
WHERE type(r) IN ['AGGRAVATES', 'CONTRAINDICATED_FOR']
RETURN h.name, type(r), n.name, r.severity, r.reason
```

Updated: Opponent Behaviour When graph_available = false *(New in v2.0)*

When `graph_available = false`, the Opponent **cannot** run its re-querying Cypher (because the herb is not in the graph). In this mode, the Opponent's system prompt instructs it to:

1. Flag ALL of the Proponent's herb-specific claims with `risk_type: "Unverified — Outside Knowledge Base"`.
2. Set `verdict: "FLAGGED"` regardless of what the Proponent said.
3. Pass control to the Judge with the instruction that no graph-verified synthesis is possible.

Why this matters: Without this, a query about an herb outside the seed dataset produces a confident-sounding Proponent draft that the Opponent cannot challenge (because it has no graph data to challenge with). The result is false confidence. With this rule, the system correctly downgrades its confidence for any out-of-scope herb.

Opponent Output Structure

```
json
{
  "verdict": "FLAGGED" | "VERIFIED",
  "graph_was_available": true | false,
  "flags": [
    {
      "claim": "Trikatu is safe for digestion",
      "risk_type": "Dosha Aggravation" | "Contraindication" | "Unverified Claim" |
"Unverified — Outside Knowledge Base",
      "severity": "HIGH" | "MEDIUM" | "LOW",
      "supporting_graph_fact": "Trikatu AGGRAVATES Pitta (strength: 4/5)",
      "relevant_patient_condition": "Patient has Acid Reflux (Pitta-associated)"
    }
  ],
  "unchallenged_claims": ["..."],
  "suggested_alternative_herbs": ["Saunf (Fennel)", "Dhania (Coriander)"]
}
```

Step 2.6 — The Judge Node + Conditional Edge Logic

LangGraph Conditional Edge (Replacing v1.0's "Early-Exit Logic")

In v2.0, the early-exit is not code logic inside the Judge — it is a **conditional edge in the LangGraph state machine**:

```
opponent_node
    |
    ▼
conditional_edge:
    ├── opponent_verdict == "VERIFIED"?
    |       → route to: output_formatter_node  (skip Judge entirely, save 1-2 seconds)
    └── opponent_verdict == "FLAGGED"?
            → route to: judge_node
```

This is declared once in the graph definition and never duplicated. It is the cleanest expression of the early-exit pattern in LangGraph.

Judge Node Behaviour

Receives three inputs from LangGraph state:

1. Original user query + user profile.
2. Proponent's structured draft.
3. Opponent's structured critique.

When Opponent flags HIGH severity with no safe alternative in graph: The Judge implements the **Graceful Degradation Protocol**:

1. Confidence Score set below 60%.
2. Response: "*Given your [Dosha] constitution and current season, no single herb in our verified knowledge base can safely address all your conditions simultaneously.*"
3. Mandatory disclaimer: "*Please consult a certified Ayurvedic practitioner (BAMS) before proceeding.*"

This is not a system failure — it is the system working correctly. It is directly citable in your paper: "*VedaAI correctly identifies when the multi-variable constraint space has no safe solution and escalates to human expert consultation.*"

When `graph_available = false` (Opponent flagged all claims as unverified): The Judge sets confidence below 50% and prepends: "*Note: This guidance is based on textual sources only. Graph-*

verified safety check was not available for [herb name]. Treat this as general information only."

Step 2.7 — Phase 2 Verification: Go/No-Go Tests

ALL tests must pass before Phase 3 begins.

Test 2.A — The Primary Trap Query Test with User Profile *(Updated in v2.0)*

Query: "*Can I take Trikatu or Ginger in summer?*" **User Profile:** `{prakriti: "Pitta", chronic_conditions: ["Acid Reflux"], current_season: "Grishma"}`

Pass Conditions:

1. Entity Extractor merges profile: `patient_conditions: ["Acid Reflux"]` appears in extracted entities even though user didn't mention it in the query.
2. Cypher Builder resolves `"Ginger"` canonically (finds it via both name and synonyms).
3. Opponent flags both herbs as contraindicated (Ushna virya + Pitta + Acid Reflux + summer).
4. Judge suggests cooling alternatives (Fennel/Saunf, Coriander/Dhania).
5. Final confidence score between 75–90% (genuine conflict, not 100%).
6. LangSmith trace shows all 5 nodes executed and their latencies.

Test 2.B — The Safe Query Test (No False Positives)

Query: "*I have dry skin and anxiety. Can I take Ashwagandha in winter?*" **User Profile:** `{prakriti: "Vata", chronic_conditions: [], pregnancy: false}`

Pass Conditions:

1. Opponent returns `"verdict": "VERIFIED"` with empty `"flags"`.
2. LangGraph conditional edge routes directly to `output_formatter_node` (Judge is NOT called).
3. Confidence score ≥ 90%.
4. Total pipeline latency under 6 seconds (verifying early-exit path works).

Test 2.C — The Hallucination Detection Test

Query: "*Is Brahmi good for memory? And also for treating COVID-19?*"

Pass Conditions:

1. Proponent marks COVID claim with `"source": "unverified"` and `[UNVERIFIED]` tag.
2. Opponent flags COVID claim as `risk_type: "Unverified Claim"`.

3. Judge separates memory benefit (graph-verified) from COVID claim and recommends former while explicitly disclaiming latter.

Test 2.D — The Zero-Result Fallback Test *(New in v2.0)*

Query: "*Can I take Kalmegh for liver support?*" *(Kalmegh is intentionally NOT in the 50-herb seed dataset)*

Pass Conditions:

1. Cypher Builder marks Kalmegh as `resolved: false`.
2. Hybrid Retriever sets `graph_available = false`.
3. Opponent flags all Kalmegh-specific claims as `"Unverified — Outside Knowledge Base"`.
4. Judge output includes: "*Kalmegh is outside our current verified knowledge base. This guidance is based on textual sources only.*"
5. Confidence score below 55%.
6. System does NOT crash. Does NOT hallucinate a confident-sounding answer.

Phase 2 Go/No-Go Decision Rule

All four tests pass + pipeline latency under 10 seconds each + LangSmith traces visible → proceed to Phase 3.

If you proceed to Phase 3 without passing these tests: You are building a UI for an AI engine with unverified reliability. The next 3 weeks of work will surface bugs that were always in the engine, but you will now spend 10x longer debugging them through the frontend abstraction layer.

PHASE 3: Full-Stack Integration (API Gateway & Frontend)

What this phase IS and IS NOT

IS: Wrapping the verified Phase 2 engine in an Express.js API, adding User Profile management (MongoDB), SSE streaming, and the Next.js chat frontend with the Transparency Inspector component. This is when the system becomes a usable product.

IS NOT: Vision Agent. Image processing is Phase 4. The file upload endpoint IS built here, but it just stores the image — it does not process it yet.

> NOTE
>
> Phase ordering changed in v2.0: Full-Stack is now Phase 3 (was Phase 4). Vision Agent is Phase 4 (was Phase 3). This is because the Vision Agent has a hard dependency on the Express.js file upload endpoint, which is built here.

Step 3.1 — API Architecture Decision

Monolith vs. Microservices vs. Monolith with Async Queue

Monolith (One Express server does everything synchronously): The LangGraph pipeline takes 4–8 seconds. Running it synchronously in an Express request handler blocks the event loop. Under 2 concurrent users, this becomes a bottleneck.

Microservices (Separate AI service, separate API gateway): Complex Docker orchestration, inter-service communication, distributed debugging. Overkill for a 4-person student team.

Monolith with Async Queue (Chosen):

- Express.js handles HTTP + WebSocket connections.
- The LangGraph pipeline runs in a **Node.js Worker Thread** (no external queue dependency).
- Results are streamed to the Next.js frontend via **Server-Sent Events (SSE)**.
- The SSE stream sends status events: `"RETRIEVING"` → `"PROPONENT_DRAFTING"` → `"OPPONENT_REVIEWING"` → `"JUDGE_SYNTHESIZING"` → `"COMPLETE"`.

Why SSE streaming is important for VedaAI's UX specifically: Your system intentionally takes 4–8 seconds because it runs a multi-step safety verification. A blank loading spinner for 7 seconds makes users think the system is broken. Streaming status events makes the wait a **feature** — users see the safety check happening in real time. This directly supports your research narrative about transparency and trustworthiness.

Step 3.2 — User Profile Schema & API *(New in v2.0)*

Why This Exists

Ayurveda is fundamentally **personalized medicine**. A stateless chatbot where users must re-enter their health context every query is both inconvenient and unsafe (users forget to mention chronic conditions).

MongoDB User Profile Schema

```
javascript
{
  userId: ObjectId,
  email: String,
  prakriti: {
    type: String,
    enum: ["Vata", "Pitta", "Kapha", "Vata-Pitta", "Pitta-Kapha", "Vata-Kapha",
"Tridoshic"]
  },
  vikruti: String,  // current imbalanced dosha
  chronic_conditions: [String],  // ["Hypertension", "Acid Reflux", "Diabetes"]
  age: Number,
  pregnancy: Boolean,
  current_season: String,  // auto-set from server date using Ayurvedic calendar
  created_at: Date,
  updated_at: Date
}
```

API Endpoints to Build

```
POST   /api/auth/register
POST   /api/auth/login
GET    /api/profile              → fetch user profile
PUT    /api/profile              → update prakriti, conditions, etc.
POST   /api/chat                 → sends query, returns SSE stream
POST   /api/upload               → stores image file (Vision Agent hooks here in Phase 4)
GET    /api/conversations        → conversation history
```

Why User Profile = Published Research Contribution

This moves VedaAI from "*an Ayurvedic chatbot*" to "*a personalized Ayurvedic guidance system with longitudinally consistent safety checking.*" You write in the paper: "*VedaAI maintains persistent user Prakriti profiles that are automatically merged with each query's entity extraction, ensuring that chronic contraindications are enforced even when the user does not explicitly mention them in the query.*" That is a concrete, measurable safety feature no competing system has.

Step 3.3 — Frontend: The Transparency Inspector Component

The Most Important UI Component in the Entire Frontend

An expandable accordion below each AI response showing:

- 🟢 **Proponent Draft:** What the initial recommendation was and which sources it drew from.
- 🔴 **Opponent Critique:** What safety concerns were flagged, which graph facts triggered them, and their severity.
- 🔵 **Judge Decision:** How conflicts were resolved, the final confidence score, and any graceful degradation messages.
- 🔵 **Graph Status:** Whether Neo4j graph facts were available for this query (from `graph_available` flag).

Why this is the most important UI element:

1. It is the **primary visual proof** that your multi-agent architecture is working — not a claim in a slide.
2. For examiners viewing the demo, clicking "Show Debate Process" and seeing a real Proponent vs. Opponent exchange is more convincing than 10 slides.
3. In your paper, screenshots of this component serve as qualitative evidence of system behaviour.
4. Without it, your demo looks identical to ChatGPT with an Ayurvedic system prompt. The examiner has no way to verify the debate is running.

Step 3.4 — Phase 3 Verification: End-to-End User Flow Test

Test 3.A — Profile-Aware Text Query

1. Create user profile: `{prakriti: "Pitta", chronic_conditions: ["Acid Reflux"]}`.

2. User sends query: "*Can I take Trikatu?*" (No mention of Acid Reflux).
3. The Entity Extractor must merge the profile and flag Acid Reflux in the entity set.
4. Opponent must flag Trikatu + Pitta + Acid Reflux contradiction.
5. Transparency Inspector shows the flag with the graph fact. *Pass condition:* Correct contraindication caught from profile data the user didn't type. Complete flow under 10 seconds. No 500 errors.

Test 3.B — SSE Streaming Test

1. Send query from browser.
2. Verify that the UI shows status messages: RETRIEVING → PROPONENT_DRAFTING → OPPONENT_REVIEWING → COMPLETE.
3. No message appears as a single block after a 7-second wait. *Pass condition:* Status events arrive progressively within 2 seconds of each pipeline stage completing.

Test 3.C — Image Upload Endpoint Test (Stub for Phase 4)

1. Upload any image file via the UI.
2. Verify it is stored in `/uploads/` directory with a unique filename.
3. Verify the API returns `{file_id: "...", status: "uploaded", vision_processed: false}`. *Pass condition:* File stored correctly. `vision_processed: false` confirms Vision Agent is not yet integrated (Phase 4).

PHASE 4: The Vision Agent (Input Seeding Layer)

What this phase IS and IS NOT

IS: A single-purpose image processing module that takes a stored image file (from Phase 3's upload endpoint) and outputs a structured JSON entity set consumed by Phase 2's Cypher Query Builder.

IS NOT: Disease diagnosis. Custom trained CNN. Computer vision research. It is an **input modality expansion** that removes the need for users to know Sanskrit herb names.

> NOTE
>
> Phase ordering changed in v2.0. Vision Agent is now Phase 4 (was Phase 3). It now builds on top of an already-working Express file upload endpoint (Phase 3) and a tested AI pipeline (Phase 2).

> Integration is a hook, not a full build.

Step 4.1 — Scope Definition

What the Vision Agent handles

IN SCOPE — Primary Use Case: Packaged Ayurvedic products (medicine bottles, churna packets, capsule boxes with printed labels). Fundamentally an OCR + semantic parsing problem. Gemini 1.5 Flash Multimodal handles this in a single API call.

IN SCOPE — Secondary Use Case (defined catalog only): Clear photographs of 10–15 common, visually distinctive herbs (Tulsi, Neem, Mint, Turmeric root, Amla berry, Shatavari root). These are visually distinctive enough for reliable multimodal LLM identification.

OUT OF SCOPE — Explicitly excluded: Wild/uncontrolled botanical photographs (varying lighting, backgrounds, drying states). This is an open hard computer vision problem. Even dedicated apps trained on millions of images (PlantNet, iNaturalist) achieve only 80–85% accuracy on uncommon species. Explicitly excluded in Section 1.5 of the extended synopsis with justification.

Why explicit scoping is more impressive than over-claiming

A system that claims to recognize all 13,000 Ayurvedic herbs and fails on basic test cases scores lower than a system that honestly scopes to 15 herbs and works reliably every time. Examiners are CS engineers — they value precision and honesty over ambition without delivery.

Step 4.2 — Implementation

Gemini 1.5 Flash Multimodal with Schema-Enforced Output

Why NOT Google Cloud Vision API (dedicated OCR): Cloud Vision extracts raw text: `"Ashwagandha Extract 300mg - Supports Stress Relief"`. You then need a second NLP step to parse "Ashwagandha" as a herb and "Stress Relief" as an indication. Two API calls, two points of failure.

Gemini multimodal does both OCR and semantic parsing in one call, outputting directly:

```
json
{
  "identified_item": "Patanjali Ashwagandha Capsules",
  "botanical_name": "Withania somnifera",
  "detected_herbs": ["Ashwagandha"],
  "packaging_indications": ["Stress", "Fatigue", "Anxiety"],
  "confidence": 0.96,
  "image_type": "packaged_product" | "fresh_herb" | "unknown"
}
```

Graph Membership Verification (Critical Safety Check)

After receiving Vision Agent output, before passing to Phase 2:

```
cypher
MATCH (h:Herb)
WHERE h.name IN $detected_herbs
   OR ANY(herb IN $detected_herbs WHERE herb IN h.synonyms)
RETURN h.name AS verified_canonical_name
```

Only `verified_canonical_names` pass to the Cypher Query Builder. Unverified names trigger: *"[Herb] was detected in the image but is not in our verified knowledge base. Text-only consultation mode activated."*

Why this check prevents a critical failure mode: Gemini may hallucinate a herb name from a blurry or ambiguous image. Without this check, the hallucinated name enters the pipeline, causes zero Cypher results, and the fallback path runs — but the user is never told the herb was unrecognized. With the check, the user receives an honest warning.

Step 4.3 — Phase 4 Verification: Go/No-Go Tests

Test 4.A — Medicine Box OCR Test

Image: Clear photo of Patanjali Ashwagandha capsule box. *Pass:* `detected_herbs: ["Ashwagandha"]`, `indications: ["Stress", "Fatigue"]`, `confidence > 0.85`.

Test 4.B — Herb Leaf Test

Image: Clear photo of Tulsi (Holy Basil) leaves. *Pass:* `identified_item: "Tulsi"`, `botanical_name: "Ocimum tenuiflorum"`, `confidence > 0.80`.

Test 4.C — Graph Membership Verification

Image: Photo of a non-Ayurvedic product (Vitamin C tablet box). *Pass:* `detected_herbs: []`. System shows warning message. Does NOT crash or hallucinate.

Test 4.D — Full Vision-to-Answer Flow *(New in v2.0)*

1. User uploads photo of Ashwagandha bottle.
2. User's stored profile: `{chronic_conditions: ["Hypertension"]}`.
3. User types: "*Is this safe for me?*"
4. Vision detects Ashwagandha → graph check verifies it → passes to Cypher Builder.
5. Profile merge adds Hypertension to entity set.
6. Opponent checks: `(Ashwagandha)-[:CONTRAINDICATED_FOR]->(Hypertension, severity: moderate)`.
7. Opponent flags. Judge warns user and recommends consulting a practitioner. *Pass:* Correct contraindication caught entirely from the image + profile. User typed no herb name. No 500 errors.

PHASE 5: Empirical Benchmarking & Academic Validation

What this phase IS

Generates the numbers that go in the research paper. Without Phase 5, you have a working system but no scientific evidence it outperforms existing approaches.

Step 5.1 — Constructing the Benchmark Dataset

The Most Critical Decision in Phase 5

Option A — LLM-generated queries: Catastrophic data contamination. If you use an LLM to generate test questions AND an LLM to answer them, results are artificially inflated. Paper rejected immediately by reviewers who recognize this.

Option B — Queries from clinical literature: Complex multi-page case studies cannot be simplified without introducing researcher bias.

Option C — Expert-curated queries (Chosen): 150 queries created manually, verified against published clinical guidelines (Charaka Samhita, AYUSH publications).

Dataset composition (Updated in v2.0):

- 75 simple queries (single herb, single symptom, no seasonal constraint).
- 75 complex queries (multi-herb, multi-dosha, seasonal constraints + planted contraindications).

> IMPORTANT
>
> **New in v2.0:** Each of the 75 complex queries must have a **ground-truth safety label** indicating whether a contraindication EXISTS in the Neo4j graph for that query. This label is required to calculate the Safety Violation Rate metric. Without it, you cannot measure whether the Opponent correctly caught the contraindication. Annotate each complex query as: `{has_contraindication: true/false, expected_flagged_herb: "Ginger", contraindication_reason: "Aggravates Pitta"}`.

Step 5.2 — Running the 3-Way Ablation Study

Three Pipelines to Test

Pipeline A (Baseline): ChromaDB only → Single LLM call. No Neo4j. No debate. **Pipeline B (Partial):** ChromaDB + Neo4j → Single LLM call. Graph-grounded but no debate. **Pipeline C (Full VedaAI):** ChromaDB + Neo4j + LangGraph Proponent-Opponent-Judge.

Why All Three Pipelines

Improvement from A→B = contribution of the Knowledge Graph alone. Improvement from B→C = contribution of the Multi-Agent Debate alone.

This is the ablation study in Section 5.4.2 of your synopsis. It is what makes the project a publishable research paper, not just a good student project.

Metrics (Updated in v2.0)

Automated Metrics (existing, unchanged):

- Precision@K, BLEU-4, ROUGE-L, BERTScore.
- Average pipeline latency (ms).

New Metric 1 — Safety Violation Rate: *Definition:* The percentage of queries where a known contraindication was present in the Neo4j graph but was NOT flagged in the final output.

Calculation: For each complex query with `has_contraindication: true`, check if the final Judge response mentions the `expected_flagged_herb` in a warning context.

```
Safety Violation Rate = (Queries with missed contraindication) / (Total queries with
contraindication) × 100%
```

Expected result: Pipeline A (Baseline) should have a high Safety Violation Rate (~40–60%). Pipeline C (Full VedaAI) should have a dramatically lower rate (~5–15%), demonstrating the Opponent Agent's value.

Why this is the most important metric in the paper: It directly measures the safety claim of your system in a way BLEU and BERTScore cannot. A health AI that gives grammatically correct but medically dangerous advice scores high on BLEU and low on Safety. Safety Violation Rate catches this.

New Metric 2 — Clinical Relevance Score (Human Evaluation): *Protocol:* Sample 30 responses randomly from each pipeline (90 total). Have an Ayurvedic practitioner (BAMS doctor, or Ayurvedic faculty member) rate each response on:

1. **Factual Accuracy** (1–5): Is the herb recommendation Ayurvedically correct?
2. **Safety Adequacy** (1–5): Are all relevant contraindications mentioned?
3. **Practical Usefulness** (1–5): Is the advice actionable for a non-expert user?

Why human evaluation is necessary: Automated metrics measure surface-level text similarity. A response that correctly identifies the right herb but uses slightly different wording will score low on BLEU. A response that sounds correct but recommends a dangerous herb in the wrong context will score high on BLEU. Only a domain expert can evaluate what actually matters for health guidance.

Step 5.3 — Statistical Significance: The Paired t-Test

For both the primary accuracy metric AND the Safety Violation Rate, run:

```
python
from scipy import stats
t_stat, p_value = stats.ttest_rel(scores_pipeline_A, scores_pipeline_C)
```

Reporting: "VedaAI (Pipeline C) demonstrates statistically significant improvement over the baseline (Pipeline A) on both retrieval accuracy ($t = X.XX$, $p < 0.05$) and Safety Violation Rate ($t = X.XX$, $p < 0.05$)."

Without the t-test, an 8% difference in accuracy could be random variation. With $p < 0.05$p < 0.05, it is proven significant. A reviewer who does not see this test will ask for it in the revision round.

Step 5.4 — Human Evaluation Protocol *(New in v2.0)*

How to Execute It

1. Sample 30 responses from each of the 3 pipelines (90 evaluation tasks total).
2. Anonymize them — the evaluator should not know which pipeline produced which response.
3. Provide the evaluator with the original query and the three rating criteria (Factual Accuracy, Safety Adequacy, Practical Usefulness).
4. Collect ratings. Calculate mean and standard deviation per criterion per pipeline.
5. Report in paper as Table X: "*Human Evaluation Results (BAMS Doctor, n=90 responses).*"

Why Anonymized Evaluation Matters

If the evaluator knows which response came from your proposed system, they will unconsciously rate it higher (experimenter bias). Anonymization ensures the evaluation is fair.

GLOBAL ARCHITECTURE: How Every Phase Connects

(v2.0)

```
Phase 1 (Data)
├── Neo4j Aura: 50+ Herb nodes (with synonyms[]), 3 Dosha, 6 Season, 25+ Symptom,
│               10+ Condition nodes
│               Relationships: PACIFIES, AGGRAVATES, TREATS, CONTRAINDICATED_FOR,
│                              ASSOCIATED_WITH, DOMINANT_IN, WORSENS_IN, COMPOSED_OF
└── ChromaDB: AYUSH + Classical text chunks → Google text-embedding-004

Phase 2 (AI Engine — LangGraph StateGraph)
├── [Node] Entity Extractor: (raw_query + user_profile) → Canonical Entity JSON
│                            (Gemini Flash + Zod schema enforcement)
├── [Step] Cypher Query Builder: Raw names → Canonical names + synonym resolution
│                                → Flags unresolvable entities
├── [Node] Hybrid Retriever: Parallel ChromaDB + Neo4j → Merged labeled context
│                            + Fallback Hierarchy if Neo4j returns 0 results
│                            → Sets graph_available flag
├── [Node] Proponent: Gemini 1.5 Pro → Structured Draft JSON
│                     (respects graph_available flag in prompt)
├── [Node] Opponent: Gemini Flash + Cypher re-querying → Critique JSON
│                    (handles graph_available = false with blanket flagging)
├── [Conditional Edge]: verdict == VERIFIED → output_formatter (skip Judge)
│                       verdict == FLAGGED  → judge_node
└── [Node] Judge: Gemini 1.5 Pro → Final Response + Confidence Score
                  (Graceful Degradation Protocol when no safe alternative exists)

Phase 3 (Full-Stack)
├── Express.js API: SSE streaming + Worker Threads + REST endpoints
├── MongoDB: User Profile Schema (prakriti, chronic_conditions, pregnancy, age)
│            + Conversation History
├── /api/upload: Stores image file. Returns {file_id, vision_processed: false}
└── Next.js Frontend:
    ├── Chat UI with SSE status event rendering
    ├── Image Upload drag-and-drop
    ├── Profile Setup page (Prakriti quiz + chronic conditions)
    └── Transparency Inspector (Proponent/Opponent/Judge accordion)

Phase 4 (Vision Agent)
└── Vision Agent: Gemini 1.5 Flash Multimodal
    ├── Input: file_id from Phase 3 upload endpoint
    ├── Output: {detected_herbs[], indications[], confidence, image_type}
    ├── Graph Membership Check: detected_herbs → verified_canonical_names
    └── Feeds canonical names → Phase 2 Cypher Query Builder

Phase 5 (Science)
```

```
├── 150-query Expert-Curated Benchmark Dataset
│   ├── 75 simple queries
│   └── 75 complex queries (each with ground-truth has_contraindication label)
├── 3-Way Ablation (Pipeline A / B / C)
├── Automated Metrics: Precision@K, BLEU-4, ROUGE-L, BERTScore,
│                      Latency, Safety Violation Rate (NEW)
├── Human Evaluation: BAMS doctor, n=90, blind evaluation, 3 criteria (NEW)
└── Statistical Tests: Paired t-test (p < 0.05) for accuracy AND Safety Violation Rate
```

Master Go/No-Go Decision Tree (v2.0)

```
Phase 1 Tests Pass? (Tests 1.A through 1.D)
  → YES: All synonym arrays populated. Multi-hop traversal verified.
         Proceed to Phase 2.
  → NO:
      1.A fails → Fix Cypher population scripts.
      1.B fails → Add synonym arrays to all Herb nodes FIRST.
      1.C fails → Reconsider embedding model or chunk size.
      1.D fails → More seed data needed.

Phase 2 Tests Pass? (Tests 2.A through 2.D)
  → YES: Profile merge works. Fallback hierarchy works.
         LangSmith traces visible. All latencies under 10s.
         Proceed to Phase 3.
  → NO:
      2.A fails → Debug entity extractor profile merge or Cypher Builder synonym
resolution.
      2.B fails → Debug LangGraph conditional edge (early-exit path).
      2.C fails → Tighten Proponent prompt to enforce [UNVERIFIED] marking.
      2.D fails → Fix Fallback Hierarchy or Opponent behaviour when graph_available=false.

Phase 3 Tests Pass? (Tests 3.A through 3.C)
  → YES: Profile-aware full flow works. SSE streaming works.
         File upload endpoint ready for Phase 4.
         Proceed to Phase 4.
  → NO:
      3.A fails → Debug profile merge path in Express → Phase 2 handoff.
      3.B fails → Fix SSE event emission timing in Worker Thread.
      3.C fails → Fix multer file upload configuration.

Phase 4 Tests Pass? (Tests 4.A through 4.D)
  → YES: Vision-to-answer full flow works.
         Graph membership check correctly filters unrecognized herbs.
         Proceed to Phase 5.
  → NO:
      4.A/B fails → Refine Gemini multimodal prompt for better OCR/herb recognition.
      4.C fails → Fix graph membership check Cypher query.
      4.D fails → Debug Vision → Cypher Builder handoff (check canonical name format).

Phase 5 Complete?
  → YES: Write extended synopsis with benchmark results, human eval table,
         and paired t-test. Submit paper.
  → NO:
      Dataset incomplete → Finish curating and labelling has_contraindication fields.
```

```
Safety Violation Rate calculation wrong → Verify ground-truth labels against Neo4j.
Human evaluation not arranged → Contact BAMS doctor / Ayurvedic faculty.
```

> **NOTE**
>
> This is Version 2.0 of the VedaAI Master Implementation Blueprint. It incorporates all 7 identified gaps from the v1.0 self-review. Changes were applied in cascade-safe order: Phase reordering (C7) first, then LangGraph (C5), Prompt Versioning (C6), Cypher Builder (C3), Fallback Hierarchy (C2), User Profile (C1), and Benchmark Metrics (C4) last. Every cross-reference, phase number, Go/No-Go test, and architecture diagram has been verified for internal consistency.

Compiled for: VedaAI — AKGEC, CSE VII Semester Project (Session 2023–2027) Document Version: 2.0 | *Revised: 2026-10-12*